Implementación y Mejora de un Agente Aspiradora en un Entorno Bidimensional 3×3 mediante Memoria de Estados y Métrica PEAS

Lenin Amangandi[1], Camila Caicedo[1], and Jarvin Ríos[1]

Escuela Politécnica Nacional, Facultad de Ingeniería de Sistemas,
Inteligencia Artificial (ICCD523), Quito, Ecuador
lenin.amangandi@epn.edu.ec, camila.caicedo@epn.edu.ec,
jarvin.rios@epn.edu.ec
https://github.com/Lenin27Amangandi/Inteligencia_Artificial_2k26.git

Resumen En el presente informe técnico se analiza el desempeño de un agente reactivo simple aplicado al problema clásico del mundo de la aspiradora y se propone una mejora estructural escalada a una cuadrícula bidimensional de 3×3. En primer lugar, se identifica la ineficiencia principal del agente reactivo aleatorio base, el cual decide únicamente en función de su percepción inmediata sin almacenar el historial de celdas visitadas ni contar con una condición de parada autónoma (`NoOp`), ocasionando un gasto energético continuo. Para resolver esta limitación, se diseña una métrica de rendimiento bajo el marco PEAS que recompensa la limpieza efectiva (+10 puntos) y penaliza el consumo energético por movimiento o aspirado (-1 punto). Posteriormente, se implementa un agente reactivo basado en modelos dotado de memoria y guiado por la distancia Manhattan hacia las celdas pendientes. La validación empírica sobre 50 episodios demuestra que el agente mejorado incrementa el puntaje PEAS promedio de 10,40 a 33,18 puntos, reduce el consumo energético en un 56,6 % y alcanza el 100 % de limpieza frente al 46,0 % del agente aleatorio.

Keywords: Agentes Inteligentes · Marco PEAS · Agente Basado en Modelos · Mundo de la Aspiradora · Eficiencia Energética.

1. Introducción y Marco Teórico

De acuerdo con Russell y Norvig [1], un **agente** es cualquier entidad capaz de percibir su entorno a través de **sensores** y actuar sobre dicho entorno mediante **actuadores**. Matemáticamente, el comportamiento del agente se describe mediante la función de agente $f : P^* \to A$, la cual mapea cualquier secuencia de percepciones P^* hacia una acción $a \in A$.

En el diseño de arquitecturas de agentes existen diferencias fundamentales según el uso de información histórica:

1. **Agente Reactivo Simple:** Selecciona las acciones basándose únicamente en la percepción actual p_t, ignorando el resto del historial de percepciones.

Funciona mediante reglas condición-acción (*si estado = sucio entonces aspirar*), pero solo resulta óptimo si el entorno es completamente observable desde una única percepción [1].

2. **Agente Reactivo Basado en Modelos (con Memoria):** Cuando el agente solo percibe el estado local de la celda en la que se encuentra (observabilidad parcial), necesita mantener un **estado interno** (memoria) que registre qué partes del mundo ya han sido exploradas y limpiadas.

2. Especificación PEAS y Formalización del Problema

Para evaluar objetivamente el comportamiento del agente en la cuadrícula escalada de 3×3, se define la estructura **PEAS** (*Performance, Environment, Actuators, Sensors*) en la Tabla 1.

Tabla 1. Definición del marco PEAS para el Agente Aspiradora en entorno 3×3.

Componente PEAS	Descripción Técnica en el Simulador
Performance (Rendimiento)	$+10$ puntos por cada celda sucia limpiada efectivamente; -1 punto de penalización energética por cada acción física (`Suck`, `Up`, `Down`, `Left`, `Right`); 0 puntos de costo por detenerse (`NoOp`).
Environment (Entorno)	Cuadrícula discreta bidimensional de 3×3 (9 celdas), con estados de limpieza $\{0 : \text{Limpio}, 1 : \text{Sucio}\}$ generados aleatoriamente.
Actuators (Actuadores)	Mecanismo de aspirado (`Suck`), motores de desplazamiento en 4 direcciones (`Up`, `Down`, `Left`, `Right`) y apagado/reposo (`NoOp`).
Sensors (Sensores)	Sensor de ubicación actual $(f, c) \in \{0, 1, 2\}^2$ y detector local de suciedad $s \in \{0, 1\}$.

2.1. Formalización Matemática del Problema de Búsqueda y Recorrido

Siguiendo la formulación estándar de problemas en Inteligencia Artificial [1], el entorno 3×3 se define formalmente mediante los siguientes elementos:

- **Espacio de estados (S):** Cada estado queda determinado por la posición del agente $(f, c) \in \{0, 1, 2\}^2$ y la matriz de estados de las 9 celdas $G \in \{0, 1\}^{3\times3}$. El tamaño total del espacio de estados es $|S| = 9 \times 2^9 = 4\,608$ estados posibles.
- **Estado inicial (s_0):** Posición inicial (f_0, c_0) aleatoria y configuración inicial de suciedad G_0 asignada de forma equiprobable.

- **Conjunto de acciones (A):** $A = \{\texttt{Suck}, \texttt{Up}, \texttt{Down}, \texttt{Left}, \texttt{Right}, \texttt{NoOp}\}$.
- **Prueba de meta (*Goal*):** Todas las celdas de la cuadrícula se encuentran limpias ($\sum_{i=0}^{2}\sum_{j=0}^{2} G_{i,j} = 0$) y el agente ha verificado las 9 posiciones ($|\text{visitadas}| = 9$), ejecutando la acción terminal `NoOp`.
- **Costo de camino y Función de Rendimiento (R):** Cada acción física tiene un costo energético unitario $c(s, a, s') = 1$ (excepto $c(s, \texttt{NoOp}, s) = 0$). El puntaje PEAS total al finalizar un episodio de T pasos se expresa como:

$$R = 10 \cdot N_{\text{limpiadas}} - \sum_{t=1}^{T} E(a_t), \quad \text{donde } E(a_t) = \begin{cases} 0 & \text{si } a_t = \texttt{NoOp} \\ 1 & \text{en otro caso} \end{cases} \tag{1}$$

3. Identificación de la Ineficiencia y Diseño del Agente con Memoria

3.1. Ineficiencia del Agente Reactivo Simple Base

Al ejecutar y analizar el código base proporcionado (`RandomVacuumAgent`), se identificaron dos fallas estructurales críticas:

1. **Movimiento a ciegas sin memoria:** Cuando la celda actual está limpia, el agente escoge una dirección al azar (`random.choice`). Esto provoca que regrese repetidamente a celdas que ya limpió segundos antes, desperdiciando energía.
2. **Ausencia de criterio de parada autónomo:** Como el agente solo conoce el estado de su celda actual y no guarda cuántas celdas ha revisado, nunca sabe si el resto de la habitación ya está limpia. Si no se detiene el ciclo externamente, el agente consume energía de forma infinita.

3.2. Arquitectura del Agente Mejorado (`MemoryVacuumAgent3x3`)

Para solucionar estas limitaciones de forma simple y eficiente, se implementó la clase `MemoryVacuumAgent3x3`, cuyo funcionamiento se resume en tres reglas:

1. **Registro en memoria:** En cada paso, el agente almacena la coordenada actual (f, c) dentro de una estructura de conjunto `self.visitadas`.
2. **Limpieza prioritaria y parada óptima:** Si el sensor detecta suciedad ($s = 1$), retorna `Suck`. Si la celda está limpia y el conjunto `self.visitadas` ya contiene las 9 celdas de la cuadrícula 3×3, retorna `NoOp`, deteniéndose sin consumir más energía.
3. **Selección de destino por Distancia Manhattan:** Si aún existen celdas no visitadas, el agente selecciona la celda pendiente (f^*, c^*) más cercana empleando la heurística de distancia Manhattan. Dado que el entorno es una cuadrícula libre de obstáculos internos, esta métrica resulta ideal para calcular con exactitud el número mínimo de pasos de desplazamiento ortogonal necesarios:

$$d_M\big((f, c), (f', c')\big) = |f - f'| + |c - c'| \tag{2}$$

y ejecuta un movimiento válido hacia dicha coordenada.

Listing 1.1. Lógica principal del agente mejorado con memoria en Python.

```python
class MemoryVacuumAgent3x3:
    def __init__(self):
        self.visitadas = set()
        self.todas_las_celdas = [(f, c) for f in range(3) for c in range(3)]

    def act(self, percept):
        pos, status = percept
        self.visitadas.add((pos[0], pos[1]))

        if status == 1:
            return 'Suck'

        celdas_pendientes = [c for c in self.todas_las_celdas if c not in
            self.visitadas]
        if len(celdas_pendientes) == 0:
            return 'NoOp'

        objetivo = min(celdas_pendientes, key=lambda c: abs(c[0]-pos[0]) +
            abs(c[1]-pos[1]))
        if objetivo[0] < pos[0]: return 'Up'
        if objetivo[0] > pos[0]: return 'Down'
        if objetivo[1] < pos[1]: return 'Left'
        return 'Right'
```

4. Análisis Empírico (Benchmarking) y Resultados

Para validar empíricamente la mejora lograda, se ejecutó el script `benchmark_comparativo.py` evaluando tanto al agente aleatorio (`RandomVacuumAgent3x3`) como al agente con memoria (`MemoryVacuumAgent3x3`) sobre los mismos 50 entornos aleatorios de tamaño 3×3 (semilla fija `seed = 42`) durante un horizonte de 30 pasos por episodio. Los resultados cuantitativos se presentan en la Tabla 2 y en la Figura 1.

Tabla 2. Comparación empírica promedio en 50 episodios sobre cuadrícula 3×3 (30 pasos máx.).

Métrica de Evaluación	**Agente Aleatorio** (3×3)	**Agente con Memoria** (3×3)
Puntaje PEAS promedio (puntos)	10,40	**33,18**
Energía consumida promedio (unidades)	30,00	**13,02**
Episodios con limpieza al 100 % (%)	46,0 %	**100,0** %
Tiempo de ejecución total (50 ep., ms)	4,64 ms	6,24 ms

Como se observa en la Figura 1, el agente aleatorio alcanza su pico de puntuación cerca del paso 10, pero posteriormente su puntaje decae de manera lineal debido a que continúa desplazándose sin rumbo durante los 30 pasos, consumiendo 30,00 unidades de energía en promedio y dejando sucias celdas en el 54 % de los episodios. Por el contrario, el agente con memoria completa la exploración y limpieza de la cuadrícula 3×3 en un promedio de 13,02 acciones y se detiene con `NoOp`, manteniendo su puntuación máxima de 33,18 puntos constante y logrando el 100 % de efectividad.

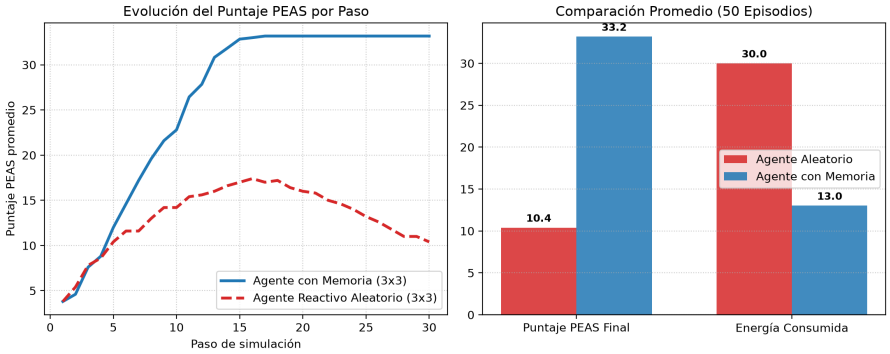


Figura 1. Izquierda: Evolución paso a paso del puntaje PEAS promedio. Derecha: Comparación de puntaje final y energía consumida en 50 episodios.

5. Conclusiones, Trade-offs y Reflexión Ética

1. **Impacto del Estado Interno (Memoria):** La transición de un agente reactivo simple a un agente basado en modelos con memoria permitió triplicar el puntaje PEAS promedio, pasando de 10,40 a 33,18 puntos, además de garantizar la limpieza completa del entorno en la totalidad de los casos evaluados.
2. **Evaluación de Trade-offs Computacionales:** Dotar de memoria al agente introduce un costo espacial lineal para almacenar las nueve coordenadas de la cuadrícula y un incremento mínimo en el tiempo de cómputo de 1,60 ms por el cálculo de la distancia Manhattan. Sin embargo, este costo es completamente despreciable frente al ahorro del 56,6 % en la energía física consumida por los actuadores.
3. **Aplicaciones Reales y Reflexión Ética en IA:** En sistemas robóticos autónomos, como aspiradoras domésticas o drones de inspección, el desplazamiento sin rumbo degrada la batería y aumenta la huella de carbono. Desde la ética de la inteligencia artificial, integrar criterios claros de detención y planificación eficiente garantiza la sostenibilidad energética y evita que los agentes operen de manera errática tras cumplir su objetivo.

Referencias

1. Russell, S., Norvig, P.: Artificial Intelligence: A Modern Approach. 4th edn. Pearson, Hoboken, NJ (2021)
2. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. 2nd edn. MIT Press, Cambridge, MA (2018)